

Why Raylib-cslo

The use of Raylib-cslo in my project stems from the fact that the base version of Raylib-cs did not include RayGUI, an essential component for the graphical interface. Additionally, Raylib-cs is only compatible with recent versions of .NET, which could lead to compatibility issues with older versions. Therefore, I opted for Raylib-cslo, which is better supported and works more smoothly with older versions of .NET.

Another reason for this choice is that Raylib-cslo integrates many additional features, such as rlgl, raygui, raymath, etc., which are extremely useful for game and graphical interface development. This saves development time by avoiding the need to integrate these libraries separately.

As for the use of WinForms, I decided not to use it because I find it not well-suited for this type of project. WinForms has limitations in terms of flexibility and control over animations and visual customization, whereas Raylib-cslo offers more freedom to create smooth animations and a more dynamic rendering.

Regarding Newtonsoft.Json, it has allowed me to better handle JSON files, which has been particularly useful for loading pieces, saves, and configuration settings. Compared to the default JSON integration in .NET 5, Newtonsoft.Json offers better flexibility and simpler management of structured data, which has saved me time and improved the handling of information in my project.

Technologies used

- [RayLib-cslo](#)
- [Newtonsoft.Json](#)

How the AI works

I was heavily inspired by this video, while making my own improvements.

<https://www.youtube.com/watch?v=DqEirMq7sD0&t>

And here is the detailed calculation used.

$$f(x, y, r) = \sum_{n=0}^N (w_1 \cdot h(b_n) + w_2 \cdot holes(b_n) + w_3 \cdot lines(b_n) + w_4 \cdot bumpiness(b_n))$$